

Yoyo Tricks with a BEANIE

Xavier Bonnetain, Sébastien Duval, Virginie Lallemand, Thierno Mamoudou Sabaly, Thomas Sagot and Thibault Sanvoisin

Université de Lorraine, CNRS, Inria, LORIA, Nancy, France

`firstname.name@loria.fr`

Abstract. BEANIE is a 32-bit tweakable block cipher, published in ToSC 2025.4, designed for memory encryption of microcontroller units. In this paper, we propose its first third-party analysis and present a key recovery against both versions. We attack the full 5+5 rounds of BEANIE using a yoyo distinguisher at a cost close to the security claim of 2^{80} time and 2^{40} data. We also propose an attack of the 7+7-round version of BEANIE, the reinforced variant with a 128-bit security claim, in time 2^{88} and with 2^{60} data.

Keywords: BEANIE · Cryptanalysis · Yoyo

1 Introduction

BEANIE [GHM⁺25] is a 32-bit low-latency tweakable block cipher designed for memory encryption. It has an AES-like structure, but with a 32-bit block size. It uses a 128-bit tweak that is encrypted with a 128-bit master key K through a PRINCE-like cipher. The 128-bit encrypted tweak is then used to derive the 32-bit round-keys with a linear key schedule, which are used to encrypt the 32-bit plaintext.

2 Specification of Beanie

There are two parts in BEANIE: a tweak-key schedule and a data path.

2.1 Data Path

The internal state of BEANIE is represented as a 4×2 matrix, where each cell contains a nibble. Cells are indexed from top to bottom and from left to right (see Figure 1). The data path is composed of r rounds, which can be written as $R' \circ R^{r-1}$. The first $r - 1$ rounds consist of the following operations:

AddRoundTweakey (ART), where the tweak-key is XORed with the internal state.

SubCells (SC), the non-linear part of the data path, where the 4-bit Sbox detailed in Table 1 is applied to each cell.

ShiftRows (SR), where the cells of the second and fourth rows are rotated one position to the left.

MixColumns (MC), where each nibble is treated as an element of $\text{GF}(2^4)/0x13$ and multiplied by an MDS involutory 4×4 matrix M . For the last round R' , the same operations are applied, except for MixColumns which is replaced with a final ART operation.

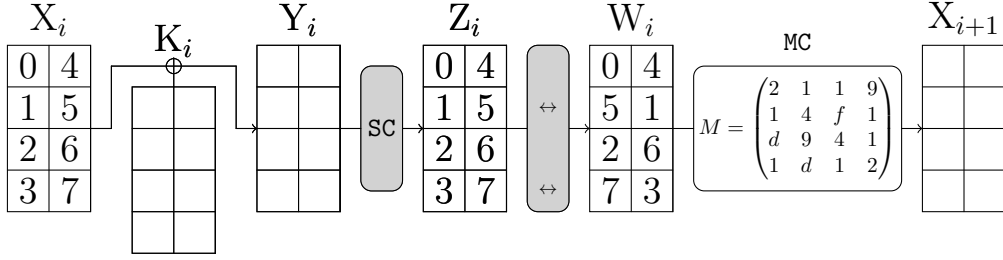


Figure 1: One round R of the BEANIE data path. State notations are taken from [GHM⁺25].

Table 1: 4-bit Sbox $\mathcal{S}$ used in BEANIE.

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$\mathcal{S}(x)$	0	4	2	b	a	c	9	8	5	f	d	3	7	1	6	e

2.2 Tweak-key Schedule

As can be seen in Figure 2, the internal state of the tweak-key schedule is represented as two 4×4 matrices, with each cell containing a nibble. Multiple operations are applied each round on these two matrices:

AddKey: the master key is XORed with the state, along with round-dependent constants.

SubCellsT (SCT’): the same 4-bit Sbox used in the data path is applied to each cell.

MixColumnsT: considering the two 4×4 matrices at the bit level (two 64-bit vectors), the result of MixColumnsT can be obtained by multiplying the vectors by a binary 64×64 matrix.

ShiftRowsT: the first row is left untouched, the second row is rotated one position to the left, the third row by two positions, and the last row by three positions.

FeistelMixT: the two matrices are decomposed into two pairs of two columns each, (X_0, X_1) and (X_2, X_3) . The result is obtained by performing the following operations: $(X_1 \oplus X_0, X_2)$ and $(X_3 \oplus X_2, X_0)$.

ShiftRowsMixT: viewing the tweak-key as a 4×8 matrix, each row is rotated to the left by 0, 2, 4, and 6 positions, respectively. After r rounds, the output is read as (k_0, k_1, k_2, k_3) , providing the round tweak-keys to XOR with the data path for the first four rounds. The values for the remaining rounds are computed as follows: $k_4 = k_0 \oplus k_1$, $k_5 = k_2 \oplus k_3$, $k_6 = k_0 \oplus k_2$, $k_7 = k_1 \oplus k_3$, $k_8 = k_0 \oplus k_3$, $k_9 = k_1 \oplus k_2$. By abuse of notation we thereafter use the term of *round-key* to refer to the *round tweak-keys*.

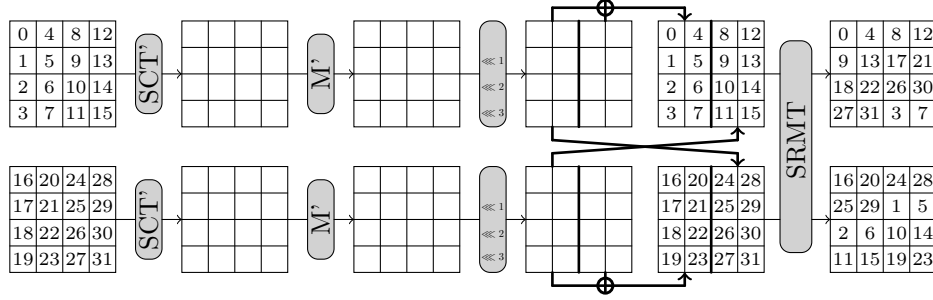


Figure 2: One round of the tweak-key schedule of BEANIE (**AddKey** is omitted for readability).

Table 2: Summary of the known attacks on BEANIE. Numbers in parenthesis are costs to recover all the round-keys. The memory is expressed in number of blocks of 32-bits.

Type	R	R Dist.	T	D	M	Reference
Boomerang	$4 + 4$	$2 + 4$	$> 2^{62}$	2^{29}	2^{30}	[GHM ⁺ 25]
	$5 + 5$	$2 + 4$	$> 2^{94}$	2^{61}	2^{30}	[GHM ⁺ 25]
Impossible diff.	$4 + 4$	$4 + 2$	$2^{73.4}$	2^{11}	$2^{21.38}$	[GHM ⁺ 25]
	$5 + 5$	$4 + 2$	$2^{105.4}$	2^{43}	$2^{21.38}$	[GHM ⁺ 25]
Impossible diff. partial coll.	$5 + 5$	$4 + 3$	2^{112}	2^{30}	$2^{27.6}$	[GHM ⁺ 25]
Integral	$4 + 4$	$4 + 2$	2^{67}	2^{18}	-	[GHM ⁺ 25]
	$5 + 5$	$4 + 2$	2^{99}	2^{50}	-	[GHM ⁺ 25]
DS-MitM	$5 + 5$	$4 + 3$	2^{112}	2^{16}	2^{109}	[GHM ⁺ 25]
Yoyo	$4 + 4$	$4 + 2$	$2^{64} (+ 2^{65})$	2^{32}	2^{32}	Subsection 5.1
	$4 + 4$	$2 + 4$	$2^{52} (+ 2^{64})$	$2^{20.75}$	$2^{34.6}$	Subsection 5.2
	$5 + 5$	$2 + 4$	2^{79}	$2^{37.3}$	2^{65}	Subsection 5.3
Truncated diff.	$7 + 7$	$2 + 2$	2^{88}	2^{60}	2^{86}	Subsection 5.4

3 Security Claim

The designers claim security in the so-called U-shape model, where queries must be of the form $(P, T_0, T_1) \mapsto E_{K, T_1}^{-1}(E_{K, T_0}(P))$. For attacks and distinguishers, the notation $r_0 + r_1$ means we target r_0 rounds of encryption followed by r_1 rounds of decryption.

In what follows we use the same notations as in [GHM⁺25] and denote by k_i , $i \in \{0, \dots, r_0\}$ the round-keys used in the encryption rounds and by $\bar{k}_j$, $j \in \{0, \dots, r_1\}$ the ones used in the decryption rounds.

For the $5 + 5$ version, the authors claim a security in 2^{80} operations and 2^{40} U-shaped queries. The authors also claim a generic 2^{128} -bit of security for the $7 + 7$ version.

4 Properties of Beanie

4.1 Subtleties of the Query Model

The U-shape model forces us to chain two copies of the cipher in our queries. This is slightly different from a standard cipher query, as the two halves are independent. In

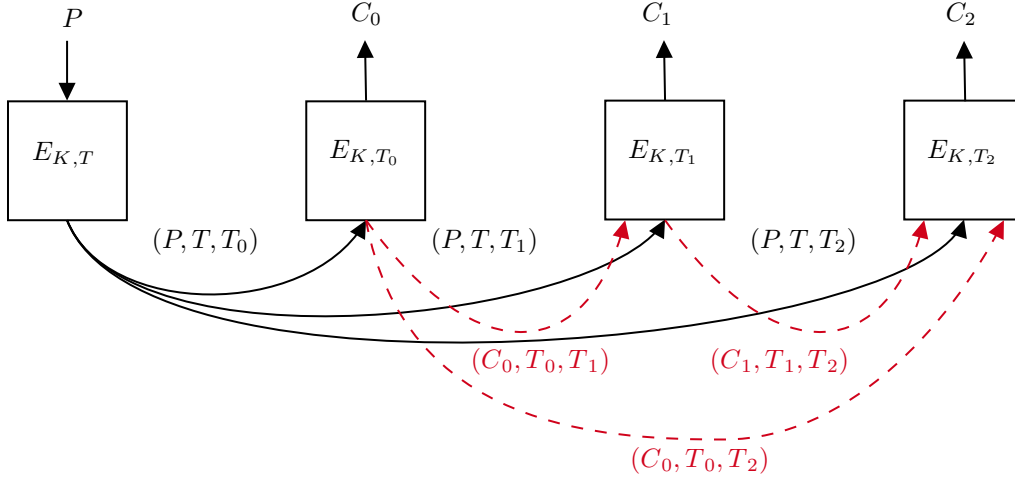


Figure 3: From 3 actual queries (in black), we can deduce 3 additional queries (in red) without actually requesting them.

particular, if C_0 is the result of the query (P, T, T_0) and C_1 is the result of the query (P, T, T_1) , then we know that C_1 would be the result of the query (C_0, T_0, T_1) :

$$(C_0, T_0, T_1) = E_{K,T_1}^{-1}(E_{K,T_0}(C_0)) = E_{K,T_1}^{-1}\left(E_{K,T_0}\left(E_{K,T_0}^{-1}(E_{K,T}(P))\right)\right) = E_{K,T_1}^{-1}(E_{K,T}(P)) = C_1$$

Thus we can compute queries we have never made if we use more than two tweaks. In the most extreme case, if we use a single plaintext and make queries of the form (P, T, T_i) for $i \in \{0, \dots, 2^{40} - 1\}$, the 2^{40} data allow to compute 2^{79} valid U-shaped queries to the cipher. This process is illustrated in Figure 3.

The designers of the cipher SCARF [CGL⁺23], which has the same attack model as BEANIE, made similar observations. In a sense, it is also reminiscent of forkciphers [ALP⁺19], where we have multiple half-ciphers and an inaccessible middle value, but with an even greater freedom in the queries we can do here.

Finding round-key collisions. The first-party analysis states that with probability 2^{-32} , the last round-key from the two tweaks are equal ($k_5 = \bar{k}_5$), meaning that the first round of decryption cancels the last round of encryption. This allows to extend an attack by 1+1 round, at a cost of a factor 2^{32} in time and data, by trying 2^{32} distinct decryption tweaks. Using the above principle, we can achieve better: decrypting the same value using $2^{16.5}$ distinct tweaks allows to obtain queries corresponding to 2^{32} distinct tweak pairs. The main constraint is that we no longer have any control on the queries; for an attack to work it needs to be in a known plaintext setting.

Limits of this approach. While we can obtain queries for many pairs of tweaks, this does not change the overall number of involved tweaks. Thus, we gain on properties that involve pairs of tweaks, but not on properties on one half of the cipher (for example, a specific value for a round-key, or a correlation on the first half).

4.2 SuperBoxes in Beanie

BEANIE is an aligned cipher, and similarly to AES, 1.5 rounds of BEANIE can be seen as the concatenation of two independent 16-bit permutations. Interestingly, the U-shape attack setting implies a similar phenomenon over more rounds in the middle of the configuration.

4.2.1 Middle SuperBox

The $4 = 2 + 2$ middle rounds of BEANIE correspond to the following series of operations (we omit the round-key additions for readability):

$$SC^{-1} \circ SR^{-1} \circ MC^{-1} \circ SC^{-1} \circ SR^{-1} \circ SR \circ SC \circ MC \circ SR \circ SC.$$

An equivalent transformation is obtained by exchanging the positions of SC and SR :

$$SR^{-1} \circ \underbrace{SC^{-1} \circ MC^{-1} \circ SC^{-1} \circ SR^{-1} \circ SR \circ SC \circ MC \circ SC}_{\hat{S}} \circ SR.$$

It appears clearly that the middle operations ($\hat{S}$) operates independently on each 16-bit column of the internal state. Expressed differently, the $4 = 2 + 2$ middle rounds of BEANIE can be seen as two independent operations that compute the nibbles 0, 2, 5, 7 of the output from the knowledge of nibbles 0, 2, 5, 7 of the input (and corresponding key material) while nibbles 1, 3, 4, 6 of the output are computed from the knowledge of nibbles 1, 3, 4, 6 of the input (and corresponding key material).

This observation allows to regard the 4 middle rounds of BEANIE as a SuperBox (as a parallel to what occurs in AES).

4.2.2 Yoyo Property on Beanie

Pushing things a step further we rewrite $4 + 2$ rounds¹ of BEANIE as follows:

$$SR^{-1} \circ \underbrace{SC^{-1} \circ MC^{-1} \circ SC^{-1} \circ SR^{-1} \circ SR \circ SC \circ MC \circ SC}_{\hat{S}} \circ \underbrace{SR \circ MC \circ SR}_{L} \circ \underbrace{SC \circ MC \circ SC}_{S} \circ SR.$$

S is a non-linear operation acting independently on each column, L is a linear operation, and $\hat{S}$ is the non-linear operation introduced previously operating independently on each column.

This is the perfect setting to apply the yoyo game introduced in [RBH17]. As a reminder, the yoyo game allows to distinguish a section of an SPN cipher if it can be rewritten as $S_2 \circ L \circ S_1$. It relies on pairs of states connected by exchange as they give states with equal Zero Difference Pattern. We recall the central notions below:

Definition 1 (Exchanged pair [RBH17]). Let $x_1 = (x_1^0, x_1^1, \dots, x_1^{w-1})$ and $x_2 = (x_2^0, x_2^1, \dots, x_2^{w-1})$ be two states of w words and $v = (v^0, v^1, \dots, v^{w-1})$ be a w -bit vector. Two states x_1' and x_2' can be derived *by exchange* from x_1 and x_2 according to the vector v as follows:

$$x_1' = \rho^v(x_1, x_2) = (x_1^0 \cdot (v^0) \oplus x_2^0 \cdot (v^0 \oplus 1), \dots, x_1^{w-1} \cdot (v^{w-1}) \oplus x_2^{w-1} \cdot (v^{w-1} \oplus 1)),$$

$$x_2' = \rho^v(x_2, x_1) = (x_2^0 \cdot (v^0) \oplus x_1^0 \cdot (v^0 \oplus 1), \dots, x_2^{w-1} \cdot (v^{w-1}) \oplus x_1^{w-1} \cdot (v^{w-1} \oplus 1)).$$

Put differently, word i of $x_1' = \rho^v(x_1, x_2)$ is equal to the corresponding word of x_1 (i.e., x_1^i) if $v^i = 1$, and is equal to the corresponding word of x_2 (x_2^i) either.

Definition 2 (Zero Difference Pattern (ZDP) [RBH17]). Let x be a state of w words. The zero difference pattern of x over its w words is given by a w -bit vector $\nu(x) = (z^0, z^1, \dots, z^{w-1})$ where $z^i = 1$ if the i -th word of x is 0, and where $z^i = 0$ either.

Theorem 1 (from [RBH17]). Let x_1 and x_2 be two states of w words and $x_1' = \rho^v(x_1, x_2)$ and $x_2' = \rho^v(x_2, x_1)$ two states deduced by exchanging words according to the vector v . Then for a $S_2 \circ L \circ S_1$ operation where S_1 is aligned on the w words we have:

$$\nu(S_2 \circ L \circ S_1(x_1) \oplus S_2 \circ L \circ S_1(x_2)) = \nu(S_2 \circ L \circ S_1(x_1') \oplus S_2 \circ L \circ S_1(x_2'))$$

¹Note that a similar reasoning allows to cover $2+4$ rounds.

Proof. The definition of the exchanged pairs implies $x_1 \oplus x_2 = x'_1 \oplus x'_2$. By applying the first Sbox layer (which is aligned with the words of the states and in particular these that have been exchanged), we deduce that

$$S_1(x_1) \oplus S_1(x_2) = S_1(x'_1) \oplus S_1(x'_2).$$

The linearity of L implies:

$$L \circ S_1(x_1) \oplus L \circ S_1(x_2) = L \circ S_1(x'_1) \oplus L \circ S_1(x'_2).$$

Finally, if some words aligned with the final Sbox application S_2 are inactive in the difference $L \circ S_1(x_1) \oplus L \circ S_1(x_2)$, the equality implies the same inactivity for $L \circ S_1(x'_1) \oplus L \circ S_1(x'_2)$. This inactivity is preserved by the bijective S_2 :

$$\nu(S_2 \circ L \circ S_1(x_1) \oplus S_2 \circ L \circ S_1(x_2)) = \nu(S_2 \circ L \circ S_1(x'_1) \oplus S_2 \circ L \circ S_1(x'_2)).$$

□

In the case of BEANIE, we consider x_1 and x_2 two states of 32 bits, and x'_1 and x'_2 two states deduced from x_1 and x_2 by exchange of columns, meaning that x'_1 has the same first column as x_1 and the same second column as x_2 , while x'_2 has the same first column as x_2 and the same second column as x_1 . Theorem 1 implies that the Zero Difference Pattern (ZDP) of $\hat{S} \circ L \circ S(x_1) \oplus \hat{S} \circ L \circ S(x_2)$ is equal to the Zero Difference Pattern of $\hat{S} \circ L \circ S(x'_1) \oplus \hat{S} \circ L \circ S(x'_2)$, that is

$$\nu(\hat{S} \circ L \circ S(x_1) \oplus \hat{S} \circ L \circ S(x_2)) = \nu(\hat{S} \circ L \circ S(x'_1) \oplus \hat{S} \circ L \circ S(x'_2)).$$

Similarly,

$$\nu(S \circ L \circ \hat{S}(x_1) \oplus S \circ L \circ \hat{S}(x_2)) = \nu(S \circ L \circ \hat{S}(x'_1) \oplus S \circ L \circ \hat{S}(x'_2)).$$

4.3 A 4+2 Round Distinguisher

The previous discussion directly leads to the distinguisher represented in Figure 4: the attacker starts from a pair of plaintexts p_1, p_2 with the same value on the first column. They modify their ciphertexts by exchanging one column inside of the pair, and ask for the corresponding decryption p'_1, p'_2 . Since $p_1 \oplus p_2$ is null on the first column, Theorem 1 implies that $p'_1 \oplus p'_2$ is also null on the first column with probability 1.

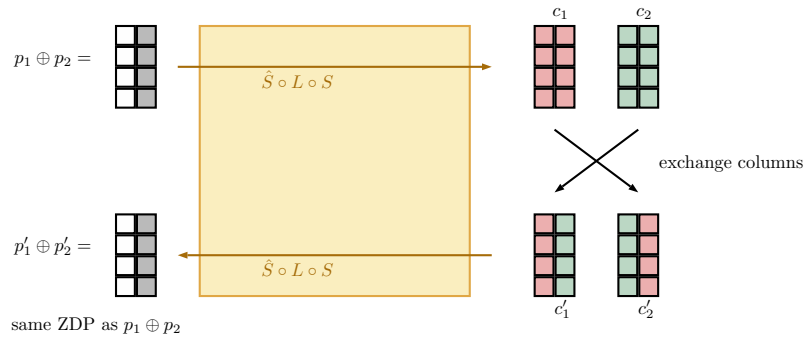


Figure 4: Direct application of the $S \circ L \circ S$ property of [RBH17].

This gives a clear distinguishing advantage, as for a random permutation this would hold with probability 2^{-16} .

Conversely, if we swap the roles of S and $\hat{S}$, we obtain a 2+4 round distinguisher.

We have experimentally confirmed this distinguisher using the reference code.

Definition 3. We call *exchange quartet* a quartet (x_1, x_2, x'_1, x'_2) made of 2 pairs (x_1, x_2) and (x'_1, x'_2) where one is an exchanged pair of the other (depending on the distinguisher, the exchange can be at the plaintext side, at the ciphertext side, or even at another point in the cipher). If one pair has an inactive column, then:

- If the other pair has the same inactive column, this is a *valid quartet*.
- Otherwise, this is an *impossible quartet* (as this cannot happen according to Theorem 1).

5 Attacks on Beanie

5.1 A First 4+4 Attack

A direct extension of this distinguisher to attack 4+4 rounds of BEANIE is explained in Algorithm 1 and Figure 5. This process shares ideas with the mixture attack from [Gra18].

Algorithm 1 Direct 4+4-round recovery of $\bar{k}_0, \bar{k}_1$ from our distinguisher.

- 1: Query the full codebook for a fixed key K and a fixed pair of tweaks (T, T') . It costs 2^{32} encryptions and memory.
 - 2: Pick a random pair of plaintexts with a difference equal to 0 in nibbles (0, 2, 5, 7).
 - 3: Guess the 64 key bits of the last 2 rounds (k_0 and k_1), and use them to build the exchange pair at the output of $\hat{S}$.
 - 4: Fetch the corresponding pair of plaintexts. Keep the key candidates that give a pair of plaintexts with inactive nibbles in positions (0, 2, 5, 7).
 - 5: Repeat for 6 initial pairs to identify the correct 64-bit key.
-

Wrong guesses. Each wrong key passes the test with probability 2^{-16} . Thus with 6 pairs, with overwhelming probability, none of the 2^{64} wrong keys survive.

Cost.

- Data: 2^{32} initial queries (full codebook).
- Time: the number of operations required is equivalent to 2^{64} encryptions.
- Memory: 2^{32} blocks of 32 bits.

5.1.1 Recovering the remainder

We can recover k_0, k_1 by swapping input and output, with the same data.

Then, assuming we have effectively identified $k_0, \bar{k}_0, k_1, \bar{k}_1$, we can recover $k_2, \bar{k}_2, k_3, \bar{k}_3$ with the generic meet-in-the-middle against double-encryption, at a cost of 2^{64} .

5.2 A second 4+4 Attack

We start by describing a 4+4 attack in which the $S \circ L \circ S$ property covers the last 6 rounds (2+4) and which is the basis for our attack on 5+5 rounds. The main difference with the attack in Section 5.1 is that we use known plaintexts instead of the full codebook, and we optimize the recovery of k_0 and k_1 , using a meet-in-the-middle technique that is similar to the one used in [DKRS20] for instance.

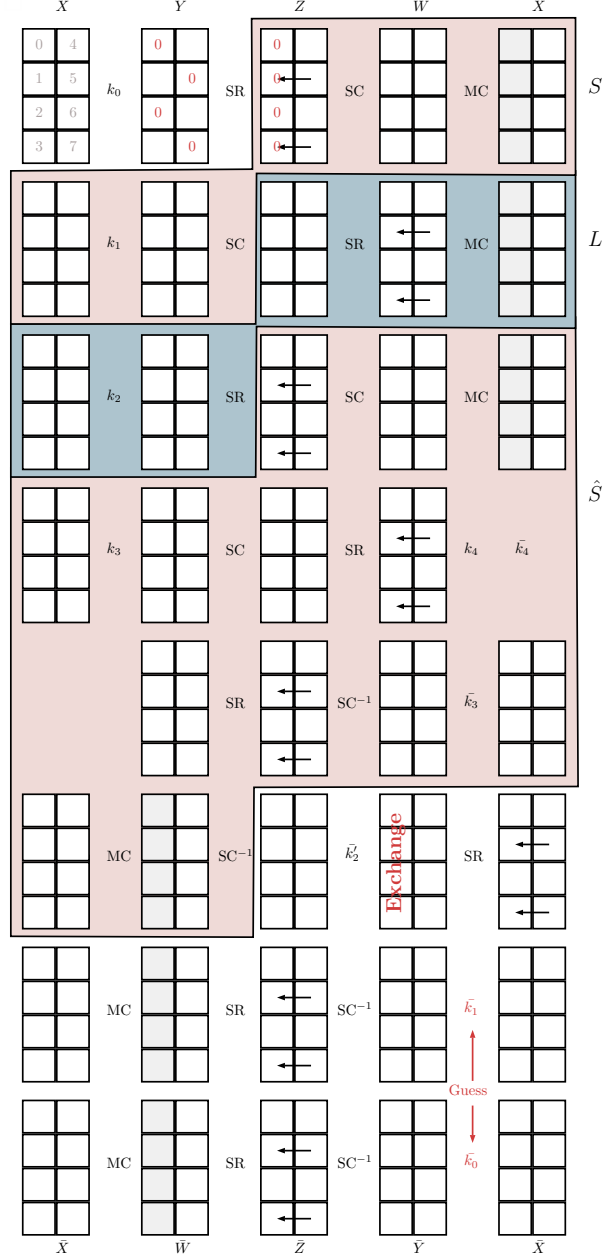


Figure 5: Attack on 4+4 rounds. $\bar{k}_2'$ is an equivalent value as the position of SR was changed.

The general idea is as follows. As explained in Algorithm 2, we start by querying random plaintexts², check the obtained ciphertexts and collect all the exchange quartets.

Next, the idea is to make guesses on the first two round-keys to count how many of the exchange quartets are valid quartets. We expect 4 valid quartets when the guessed key is correct, and strictly less when the guessed key is incorrect. As detailed in Algorithm 3,

²A version of the attack relying on chosen plaintexts is possible, but the known plaintext scenario is required to extend the attack to 5+5 rounds.

we use the fact that the first 1.5 rounds of BEANIE can be seen as two independent 16-bit functions (S^L computes $Z_1[0, 1, 2, 3]$ from $X_0[0, 2, 5, 7]$ and from 32 bits of k_0 and k_1 , as shown in Figure 6, and similarly S^R computes $Z_1[4, 5, 6, 7]$ from $X_0[1, 3, 4, 6]$ and from the other 32 key bits of k_0 and k_1). The key guess of 64 bits is thus managed as two independent key guesses of 32 bits that are next recombined together.

5.2.1 Properties of the Distinguisher

The general principle of the attack is to identify random exchange quartets at the ciphertext side, and then look for valid quartets. Due to the distinguisher, for a random quartet and a fixed position for the inactive column this holds with probability 2^{-16} for the correct key guess, and 2^{-32} for wrong guesses. Moreover, it is impossible that the correct key guess leads to a quartet in which only one pair has an inactive column and not the other.

That is:

- For the correct key guess, a quartet is valid with probability 2^{-15} .
- For a wrong key guess, a quartet is valid with probability 2^{-31} .
- For a wrong key guess, a quartet is impossible with probability 2^{-14} .

5.2.2 Attack

Algorithm 2 Construction of random exchange quartets. We only use fixed tweaks, known-plaintext queries.

- 1: Query 2^c random plaintexts for a fixed key K and a fixed pair of tweaks (T, T') .
 - 2: **for all** 2^{2c-1} pairs **do**
 - 3: Check if both ciphertexts of the exchange pair are among the random ciphertexts we have.
 - 4: Store the exchange quartet if this is the case.
 - 5: **end for** ▷ Fix $c = 20.75$ to get 4×2^{15} exchange quartets
-

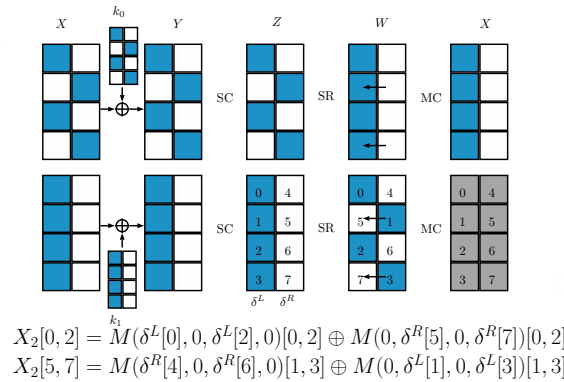


Figure 6: The first 1.5 rounds of BEANIE can be seen as the application of two independent functions: S^L (applying on the blue nibbles of the figure) and S^R (applying on the white nibbles). The attacker uses this independence to compute the difference in $X_2[0, 2, 5, 7]$ (resp. in $X_2[1, 3, 4, 6]$) by combining values obtained with independent guesses on 32 key bits.

Algorithm 3 Key-recovery of k_0, k_1 . The collisions are detailed in Figure 6.

- 1: **for all** exchange quartets (p_1, p_2, p'_1, p'_2) **do**
 - 2: **for all** 32 bits of k_0, k_1 involved in the first 16-bit function S^L **do**
 - 3: Compute S^L on p_1 and p_2 and deduce their contribution to the difference in nibbles 0, 2, 5 and 7 of X_2 , that is $M(\delta^L[0], 0, \delta^L[2], 0)[0, 2]$ and $M(0, \delta^L[1], 0, \delta^L[3])[1, 3]$. Similarly, compute their contribution to the difference in nibbles 1, 3, 4 and 6 of X_2 .
 - 4: Compute S^L on p'_1 and p'_2 and deduce their contribution to the difference in nibbles 0, 2, 5 and 7 of X_2 , that is $M(\delta'^L[0], 0, \delta'^L[2], 0)[0, 2]$ and $M(0, \delta'^L[1], 0, \delta'^L[3])[1, 3]$. Similarly, compute their contribution to the difference in nibbles 1, 3, 4 and 6 of X_2 .
 - 5: Store the concatenated values in Table T_1 together with the key guess.
 - 6: **end for**
 - 7: Follow the same idea with the second 16-bit function S^R and store the results in a table T_2
 - 8: Sort both arrays according to the differences in nibbles 0, 2, 5 and 7 and look for collisions on these values (corresponding to a valid quartet for a given key guess.)
 - 9: Sort both arrays according to the differences in nibbles 1, 3, 4 and 6 and look for collisions on these values (corresponding to a valid quartet for a given key guess.)
 - 10: **end for**
 - 11: For all keys with the maximum number of valid quartets, test if there is an impossible quartet and discard the key guess if there is one.
 - 12: Keep the surviving keys with the largest number of valid quartets.
-

Wrong guesses. The correct key is expected to have 4 valid quartets (the inactivity can be either in the first or in the second column, so we have a probability of 2^{-15}).

The probability for a wrong key to have 4 valid quartets and no impossible quartet is $\binom{2^{17}}{4} \times (2^{-31})^4 \times (1 - 2^{-14})^{2^{17}-4} \simeq 2^{-72.1}$. As we test a total of 2^{64} keys we do not expect any wrong key guess to survive.

Overall cost. Finding collisions requires sorting lists, so as for all the 2^{17} quartets we sort 4 times lists of size 2^{32} it is equivalent to 2^{56} lookups. Moreover, we do 2^{51} 2-round encryptions, and use $2^{20.75}$ data. Thus the attack costs around 2^{52} time. Memory complexity is dominated by the two arrays to find collisions, which are arrays of 96-bit words of length 2^{32} .

5.2.3 Recovering the remainder

As with the previous attack, we can recover the other round-keys at a cost of 2^{64} .

5.3 Attacking 5+5 Rounds

We will apply the previous attack to 2^{32} unordered pairs (T_i, T_j) and hope to find a collision $k_5 = \bar{k}_5$. As was indicated in the first-party analysis, this collision generically allows to add 1+1 round to any distinguisher or attack, as the 2 middle rounds will cancel each other.

5.3.1 Queries

We use $c = 20.8$ to obtain 5×2^{15} exchange quartets. We choose a fixed encryption tweak T and $2^{16.5}$ random decryption tweaks T_i . We query the same 2^c random plaintexts for these $2^{16.5}$ (T, T_i) , so the data requirement of the attack is $2^{37.3}$. As explained in Section 4.1, the initial queries give us access to 2^c plaintext/ciphertext pairs for *each* of the 2^{32} unordered pairs (T_i, T_j) . We expect 5×2^{15} exchange quartets for each tweak pair, and we can apply the previous attack.

5.3.2 Precomputation

To keep the time complexity under the security claim of the designers, the idea is to avoid repeating the array sorting process (Line 8 and 9 of Algorithm 3) by preparing precomputation tables. As explained in Algorithm 4, we precompute the list of output differences that are obtained at the output of the 16-bit function $\mathcal{S}^L$ for any possible pair of 16-bit input values and any possible 32-bit key material. As the 16-bit functions $\mathcal{S}^L$ and $\mathcal{S}^R$ do the same operations, these precomputations can be used for both. In order to have arrays directly usable to check for an inactive column we prepare 3 arrays per pair of 16-bit inputs:

- An array D that contains the output difference for all 2^{32} keys, indexed by the key,
- An array T_1 that contains the output difference and the key, sorted by the contribution of $\mathcal{S}$ to the first column through the MixColumns,
- An array I_1 that contains the first index of appearance of a given contribution in T_1 ,
- An array T_2 that contains the output difference and the key, sorted by the contribution of $\mathcal{S}$ to the second column through the MixColumns,
- An array I_2 that contains the first index of appearance of a given contribution in T_2 .

Algorithm 5 details how we use these precomputed tables to reconstruct the collision arrays we need.

Algorithm 4 Construction of random exchange quartets. We only use fixed tweaks, known-plaintext queries.

```

1: for all  $2^{31}$  pairs of 16-bit plaintexts do
2:   for all  $2^{32}$  possible keys  $k_0, k_1$  involved in the 16-bit function  $\mathcal{S}^L$  do
3:     Compute the output difference  $\alpha^L$ 
4:     Store the output difference in an array:

$$D[k_0, k_1] = \alpha^L$$

5:     Create two arrays  $T_1, T_2$  storing the output difference and the key

$$T_1[k_0, k_1] = T_2[k_0, k_1] = (\alpha^L, k_0, k_1)$$

6:   end for
7:   Sort  $T_1$  according to the differences relevant for  $\mathcal{S}^L$  to determine whether  $X_2[0, 2, 5, 7]$ 
   is inactive and sort  $T_2$  according to the ones for  $X_2[1, 3, 4, 6]$ .
8:   Compute  $I_1$  and  $I_2$  with a sequential read of  $T_1$  and  $T_2$ 
9: end for

```

In Line 7 of Algorithm 4, the array T_1 is sorted according to

$$\alpha = M(\delta[0], 0, \delta[2], 0)[0, 2] || M(0, \delta[1], 0, \delta[3])[1, 3],$$

which is relevant for $\mathcal{S}^L$ to check if $X_2[0, 2, 5, 7]$ is inactive and for $\mathcal{S}^R$ to check if $X_2[1, 3, 4, 6]$ is inactive,

and T_2 is sorted according to

$$\alpha = M(0, \delta[1], 0, \delta[3])[0, 2] || M(\delta[0], 0, \delta[2], 0)[1, 3],$$

which is relevant for $\mathcal{S}^L$ to check if $X_2[1, 3, 4, 6]$ is inactive and for $\mathcal{S}^R$ to check if $X_2[0, 2, 5, 7]$ is inactive.

In the end we obtain two sorted tables for each input pair, with an overall size of 2^{63} . This costs 2^{69} , as we are sorting many arrays of size 2^{32} .

5.3.4 Cost Estimate

Constructing a collision table costs 3 memory accesses per quartets and keys, plus one for the actual collision search. Thus the collision part has a unitary cost of 8 lookups, that is, $2^{32+16+32+3} = 2^{83}$ overall. All the parts after the collision are negligible, as only a limited number of key guesses remain.

The attack uses $2^{37.3}$ data, and the dominant cost is $2^{32+16.6+32+3} = 2^{83.6}$ lookups. There is no direct equivalence between lookups and encryption; if for example we estimate that a lookup of an array with 2^{32} elements costs as much as an Sbox layer over 32 bits and neglect the cost of the other cipher operations, then we can estimate that one 32-bit lookup costs 1/50th of a U-shaped query (if we include the tweakkey-schedule). Thus we will somewhat arbitrarily round our lookup costs to 1/32th of a U-shaped query.

Under this assumption, the overall cost is around 2^{79} . Other estimates could make the attack go above 2^{80} , but in all cases the cost is roughly at the bound.

5.3.5 Recovering the Remainder

This attack only recovers round-keys associated to a pair of random tweaks. Still, as is stated in the first-party analysis, this is sufficient: given the example attacks in the first-party analysis, we can expect to recover the round-keys for the $2^{16.5}$ tweaks we queried from the ciphertexts that we can now compute, and finally recover K by attacking the tweak-key schedule at a comparably negligible cost.

5.3.6 Overall Cost

The steps after k_0, k_1 have been recovered have a comparably negligible cost. Thus, overall, our attack costs $2^{37.3}$ data, $2^{83.6}$ lookups that we estimate cost 2^{79} overall, and uses 4 arrays of 2^{63} cells, which corresponds overall to an equivalent 2^{65} memory.

5.3.7 Metric Considerations

Our attack on the full cipher has a cost very close to the security claim. Moreover, it depends on how we compare a lookup to an encryption. As here we consider a cipher designed for encrypting lookups, at the very least the attack would not beat the security claim if it ran on a computer that uses this very cipher to secure its memory. We can also note that we do not distinguish read, write, sequential and random memory accesses in our estimates. Still, the security claim is not tailored with a generic bound, but for practicality. Thus, there is no simple attack with almost the same cost. Moreover, our attack has some advantages:

- It can be parallelised, as the 2^{32} tweak pairs can be tested independently (note that the precomputation can be reduced or removed at only a mild increase in the cost, depending on the exact cost metric),
- It uses known-plaintext chosen-tweak queries, which matches well the concrete attack scenarios for this cipher.

Finally, if we assume a determined nation-state (or worldwide company) can invest 2^{80} resources to break the cipher, an estimate slightly above the bound only means that a coalition of a few determined nation-states can break it.

5.4 Attacking 7+7 Rounds

5.4.1 Round-Key Collisions

As noted by BEANIE’s designers and as recalled in Section 4.1, when a last round round-key collision occurs, the two middle rounds cancel each other and the attacker is left with a

reduced version of BEANIE to attack.

In the case of an analysis of the 7-round version, an attacker would look for a collision between k_7 and $\bar{k}_7$, k_6 and $\bar{k}_6$, and so on so forth to cancel several middle rounds. While the first 4 round-keys are built from a process based on the PRINCE cipher [BCG⁺12] the next round-keys are deduced linearly by: $k_4 = k_0 \oplus k_1$, $k_5 = k_2 \oplus k_3$, $k_6 = k_0 \oplus k_2$, $k_7 = k_1 \oplus k_3$.

Remarkably, these simple relations imply a collision between k_4 and $\bar{k}_4$ if the last 3 round-keys collide ($k_5 = \bar{k}_5$, $k_6 = \bar{k}_6$ and $k_7 = \bar{k}_7$). This observation induces that by checking around 2^{96} pairs of tweaks in the hope of a collision between the 3 last round-keys we actually observe a collision between the 4 last round-keys, reducing 7+7 BEANIE to 3+3 BEANIE.

As the security claim of 7+7 BEANIE is of 128 bits, expecting such a collision is possible, albeit leaving a limited margin for the 3+3-round attack. We show how to handle this next.

5.4.2 7+7-Round Attack

Overall idea. This attack relies on a 2+2-round truncated differential distinguisher of probability 1 deduced from the observation made in Section 4.2.1. As 2+2 middle rounds of BEANIE can be seen as two independent operations on 16 bits, having no difference on nibbles 0, 2, 5, 7 of the input implies that nibbles 0, 2, 5, 7 of the output are inactive.

To attack 3+3 rounds, the overall idea is to independently make a guess on k_0 and $\bar{k}_0$ and to next put together the guesses leading the same data pairs to valid inputs and outputs for the truncated differential distinguisher.

To limit the required data complexity, we again take advantage of the query model (see Section 4.1).

Detailed description of the attack. We start by querying the encryption of 2^p plaintexts P_i for a fixed key K and a set of 2^t pairs of tweaks (T, T_α) where T is fixed and T_α takes 2^t different values. We denote by C_i^α the ciphertext corresponding to the encryption of P_i with the key K and the tweak (T, T_α) .

As noted in Section 4.1 this initial collection of data can be seen as 2^{2t} sets of 2^p plaintexts with their associated ciphertext when encrypted under key K and tweak (T_α, T_β) , denoted (C_i^α, C_i^β) .

For each tweak T_α , and for all the 2^{2p-1} unordered pairs C_i^α, C_j^α we want to compute the set of first round round-keys that make this pair inactive in nibbles 0, 2, 5, 7 after one round. To do so efficiently we use some precomputations as explained in Algorithm 7. The idea is to start from the 2^{48} possible pairs of internal states (X_1, X_2) positioned after the MixColumns of the first round and for which $X_1 \oplus X_2$ is inactive in nibbles 0, 2, 5, 7. We compute their decryption over MC, SR and SC and obtain the associated set of 2^{48} possible pairs of internal states (Y_1, Y_2) positioned after the addition of k_0 . We store Y_1 at the index $Y_1 \oplus Y_2$ in a table A .

Now, given a pair of plaintexts (P_1, P_2) we know that the set of possible keys k_0 that would lead to the desired inactivity pattern is equal to $\{P_1 \oplus Y | Y \in A[P_1 \oplus P_2]\}$.

Algorithm 7 Array precomputation

- 1: Initialize an array A of 2^{32} empty lists
 - 2: **for all** (X_1, X_2) such that $X_1 \oplus X_2$ is inactive in nibbles 0, 2, 5, 7 **do**
 - 3: Compute Y_1, Y_2 the keyless one-round decryption of X_1 and X_2
 - 4: Add Y_1 to the list $A[Y_1 \oplus Y_2]$
 - 5: **end for**
-

We next build a table where each line contains the tweak T_α , the 32-bit key guess for k_0 , and the set of pairs of indices (i, j) corresponding to C_i^α, C_j^α with a difference inactive in nibbles 0, 2, 5, 7 after one round when the first-round key is k_0 . The table is sorted according to the list of pairs of indices (i, j) .

A collision of the exact list of pairs of indices (i, j) indicates a behaviour in line with the probability-one truncated differential trail in the middle, and hints a candidate key pair.

Analysis of the attack. The attacker initially queries 2^{p+t} encryptions.

Building the table requires for each tweak T_α to go over all the 2^{2p-1} pairs, and for each one to write down the expected 2^{16} keys for which this pair is inactive in nibbles 0, 2, 5, 7 after one round.

The table has a total of 2^{t+32} lines, so sorting it requires $(t+32) \times 2^{t+32}$ operations. For a given tweak T_α and a given key guess of k_0 , a proportion of 2^{-16} pairs has a difference inactive in nibbles 0, 2, 5, 7 after one round, so in each line of the table we expect an average of $2^{2p-1-16}$ pairs of indices (i, j) .

For the correct pair of tweak T_α , T_β that gives the 4 last round-keys colliding and for the correct key guess of k_0 , $\bar{k}_0$ we have the exact same list of pairs of indices (i, j) in the table. We of course need to choose p such that this number of indices is greater than 0, which gives the constraint $p > 8.5$. The number of considered tweak pairs must be sufficient to obtain the desired collisions for the 3 last round round-keys, so we need $t > 48.5$.

To finalise the analysis, we need to determine the number of false positive results we might obtain. The easier way to approximate this requires to make the assumption that in each line of the table there is exactly 2^{2p-17} pairs of indices (i, j) . We want to approximate the number of pairs of (different) tweaks T_α, T_β associated to key guesses for $k_0, \bar{k}_0$ that collide within this list. The number of possible lists of 2^{2p-17} pairs is equal to $L = \binom{2^{2p-1}}{2^{2p-17}}$ so the probability to collide is $1/L$. Given that we have $2^{2t-1+64}$ candidates for a tweak pair associated to a key guess pairs, we expect $2^{2t-1+64} \times 1/L$ false positive elements.

By setting $t = 49$ and $p = 11$, the probability of having a false positive elements is around 2^{-393} and we expect to find 2 pairs of colliding tweaks on average.

The data complexity of the attack is 2^{60} . It requires to sort a table of size 2^{81} , and the correct tweak pairs associated with the correct key guesses is detected by colliding on a list of 2^5 pairs of indices.

Improved complexities could be obtained by fine-tuning further the number of pairs of indices on which we collide or by considering the other truncated differential of probability 1 (nibbles 1, 3, 4, 6).

Given that we are far from the bound (2^{128}) we simply estimate the cost to be 2^{88} , which assumes that one step of sorting (which involves a few memory accesses and a comparison) costs at most one encryption.

Algorithm 8 Attack on the 7+7-round version of BEANIE

- 1: Query 2^p random plaintexts for a fixed key K and 2^t pairs of tweaks (T, T_α) .
 - 2: **for all** 2^t tweaks (T_α) and every 2^{2p-1} pairs of plaintext **do**
 - 3: Computes the set of first round round-keys that make this pair inactive in nibbles 0, 2, 5, 7 after one round using table A from Algorithm 7.
 - 4: Store these results in a table, where each line is $T_\alpha, k_0, (i, j)$.
 - 5: **end for**
 - 6: Sort the table according to the list of pairs of indices (i, j) .
 - 7: A collision in the table gives a candidate for the keys $k_0, \bar{k}_0$ and for the pair of tweaks colliding on k_4, k_5, k_6 and k_7 .
-

Acknowledgments

We thank the designers of BEANIE for their feedback on the early version of this document. This work has been partially supported by the French Agence Nationale de la Recherche through the OREO project under Contract ANR-22-CE39-0015, and through the France 2030 program under grant agreement No. ANR-22-PECY-0010 CRYPTANALYSE.

References

- [ALP⁺19] Elena Andreeva, Virginie Lallemand, Antoon Purnal, Reza Reyhanitabar, Arnab Roy, and Damian Vizár. Forkcipher: A new primitive for authenticated encryption of very short messages. In Steven D. Galbraith and Shihō Moriai, editors, *ASIACRYPT 2019, Part II*, volume 11922 of *LNCS*, pages 153–182. Springer, Cham, December 2019.
- [BCG⁺12] Julia Borghoff, Anne Canteaut, Tim Güneysu, Elif Bilge Kavun, Miroslav Knežević, Lars R. Knudsen, Gregor Leander, Ventzislav Nikov, Christof Paar, Christian Rechberger, Peter Rombouts, Søren S. Thomsen, and Tolga Yalçın. PRINCE - A low-latency block cipher for pervasive computing applications - extended abstract. In Xiaoyun Wang and Kazue Sako, editors, *ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 208–225. Springer, Berlin, Heidelberg, December 2012.
- [CGL⁺23] Federico Canale, Tim Güneysu, Gregor Leander, Jan Philipp Thoma, Yosuke Todo, and Rei Ueno. SCARF - A low-latency block cipher for secure cache-randomization. In Joseph A. Calandrino and Carmela Troncoso, editors, *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023*, pages 1937–1954. USENIX Association, 2023.
- [DKRS20] Orr Dunkelman, Nathan Keller, Eyal Ronen, and Adi Shamir. The retracing boomerang attack. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 280–309. Springer, Cham, May 2020.
- [GHM⁺25] Simon Gerhalter, Samir Hodžić, Marcel Medwed, Marcel Nageler, Artur Folvaczny, Ventzi Nikov, Jan Hoogerbrugge, Tobias Schneider, Gary McConville, and Maria Eichlseder. BEANIE – a 32-bit cipher for cryptographic mitigations against software attacks. *IACR Transactions on Symmetric Cryptology*, 2025(4), 2025.
- [Gra18] Lorenzo Grassi. Mixture differential cryptanalysis: a new approach to distinguishers and attacks on round-reduced AES. *IACR Trans. Symm. Cryptol.*, 2018(2):133–160, 2018.
- [RBH17] Sondre Rønjom, Navid Ghaedi Bardeh, and Tor Helleseeth. Yoyo tricks with AES. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 217–243. Springer, Cham, December 2017.

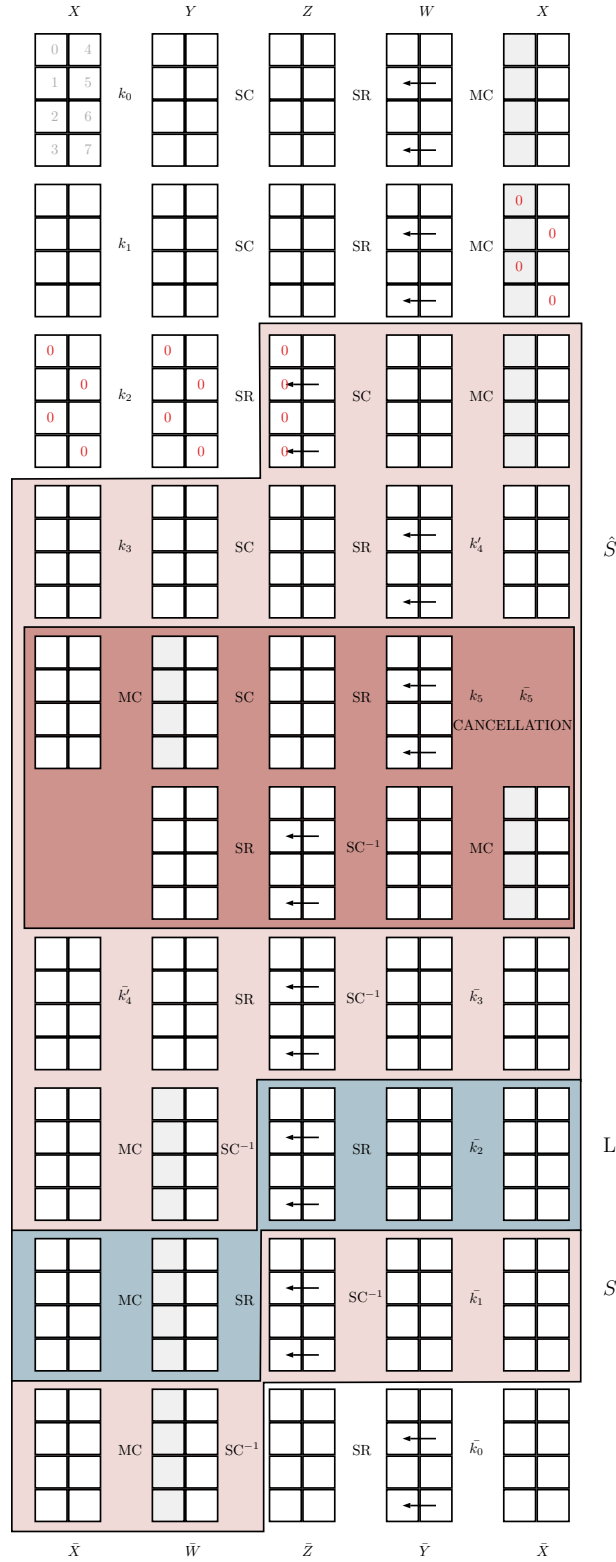


Figure 7: Attack on 5+5 rounds of BEANIE. k'_4 and $\bar{k}'_4$ are equivalent values.